

Stable Haskell

Julian Ospald

June, 2026

Introduction

Follow the presentation on your device



Stable Haskell



Why Stable Haskell?

Have you ever wondered:

▶ why can't I run `cabal install ghc`?

Why Stable Haskell?

Have you ever wondered:

- ▶ why can't I run `cabal install ghc`?
- ▶ why can't I update to a new GHC and compile my existing codebase?

Why Stable Haskell?

Have you ever wondered:

- ▶ why can't I run `cabal install ghc`?
- ▶ why can't I update to a new GHC and compile my existing codebase?
- ▶ why are cross compilers so hard to use?

Why Stable Haskell?

Have you ever wondered:

- ▶ why can't I run `cabal install ghc`?
- ▶ why can't I update to a new GHC and compile my existing codebase?
- ▶ why are cross compilers so hard to use?
- ▶ why is contributing to GHC so hard?

Our vision

We want to create a Haskell experience that is:

- ▶ stable
 - ▶ language, compiler
 - ▶ installation, updates
 - ▶ platform support
- ▶ focuses on the end user
 - ▶ releases are for end users
 - ▶ the user is in control
- ▶ allows rapid development and backporting
- ▶ has a clean foundation

What is Stable Haskell?

- ▶ a fork of GHC and Cabal
 - ▶ follows the latest LTS release (9.14 currently)
 - ▶ regular attempts at upstreaming changes
 - ▶ some changes are too radical to upstream (yet)
- ▶ a project that tries to identify and solve the missing pieces
 - ▶ we're also happy if someone else implements them
 - ▶ we engage rigorously with upstream projects
 - ▶ we're ok to deviate too
 - ▶ communication and raising awareness

Who is stable Haskell?

Our team includes the engineers who built and contributed to:

- ▶ GHC's Native Bignum Backend — Arbitrary-precision integers
- ▶ AArch64 Native Code Generator — ARM support in GHC
- ▶ JavaScript Backend in GHC — Haskell in the browser
- ▶ GHCup — The standard Haskell toolchain installer
- ▶ haskell.nix — Reproducible Haskell builds with Nix
- ▶ Cross-compilation support — Windows, iOS, Android targets
- ▶ GHC's In-Memory Loader & Linker — Dynamic loading infrastructure
- ▶ ...

The GHC project

GHC and stable haskell

- ▶ Stable Haskell didn't come into existence in a vacuum
- ▶ we're not bored
- ▶ there are non-technical reasons for the project too

GHC: A success story

- ▶ lots of experts work on GHC
 - ▶ SPJ is still active after many decades
 - ▶ relentless contributions by many companies
- ▶ focused on open collaboration
 - ▶ the academic approach
 - ▶ GHC steering committee
- ▶ a driver for innovation
 - ▶ dependent haskell
 - ▶ linear types
- ▶ wide adoption in industry

The dark side of GHC

- ▶ lack of stability

The dark side of GHC

- ▶ lack of stability
- ▶ engineering vision/priorities are often unclear

The dark side of GHC

- ▶ lack of stability
- ▶ engineering vision/priorities are often unclear
- ▶ poor contribution experience

The dark side of GHC

- ▶ lack of stability
- ▶ engineering vision/priorities are often unclear
- ▶ poor contribution experience
- ▶ communication issues

The dark side of GHC

- ▶ lack of stability
- ▶ engineering vision/priorities are often unclear
- ▶ poor contribution experience
- ▶ communication issues
- ▶ too tight coupling between systems/projects

The dark side of GHC

- ▶ lack of stability
- ▶ engineering vision/priorities are often unclear
- ▶ poor contribution experience
- ▶ communication issues
- ▶ too tight coupling between systems/projects
- ▶ driving change is hard

The dark side of GHC

- ▶ lack of stability
- ▶ engineering vision/priorities are often unclear
- ▶ poor contribution experience
- ▶ communication issues
- ▶ too tight coupling between systems/projects
- ▶ driving change is hard
- ▶

Lack of stability

Upgrading GHC is costly.

- ▶ GHC ships with changes to base and boot libraries
 - ▶ user can't opt out
 - ▶ cabal solver is peculiar #9669
- ▶ GHC ships with breaking changes to the surface language
 - ▶ simplified subsumption
 - ▶ changes in warnings/error flags
 - ▶ GHC stability principles
- ▶ Breaking changes to the bindist
- ▶ Breaking changes to the build system
- ▶ Platforms sometimes come and go (FreeBSD)

Engineering vision/priorities

- ▶ technical debt accumulates steadily
 - ▶ who keeps track of how bad it is?
 - ▶ everyone just works in isolation on different issues
 - ▶ => lack of holistic systems view
- ▶ things develop “organically” instead of strategically
- ▶ the “if we had infinite funding” argument
- ▶ unclear how things are prioritized
- ▶ a possible structure of documenting goals and progress
 - ▶ do vs want vs can

Poor contribution experience

See #26728.

Poor contribution experience

See #26728.

Case study Support statically linking executables properly:

- ▶ maybe took half a week to implement
- ▶ took ~3 months to get merged
- ▶ still not backported to 9.14, because we missed the barely communicated deadline

Communication issues

- ▶ staying in the loop is hard
- ▶ who are even the stakeholders?
- ▶ getting involved in releases is confusing
- ▶ why are there no calls for help
 - ▶ FreeBSD?
- ▶ but: some progress wrt release communication

Tight systems coupling

- ▶ GHC and cabal
 - ▶ both are handling linking
 - ▶ GHC knows how to link against already compiled packages
 - ▶ cabal figures out how to link the *current package*
 - ▶ package.conf format
 - ▶ jsem
- ▶ GHC and base
 - ▶ only way to ship new base is to bump it in GHC
 - ▶ boot libs need patches during a GHC release
- ▶ GHC and hadrian
 - ▶ is built for GHC and GHC only
 - ▶ is full of hacks and ad-hoc logic
- ▶ GHC and gitlab

=> decisions are often driven by what is convenient for GHC

Driving change is hard

- ▶ how to gather consensus
 - ▶ who is responsible?
 - ▶ who is the domain expert?
- ▶ contributing is unpleasant and below current open source standards
 - ▶ despite there being a helpful team of GHC developers
- ▶ general risk aversion
 - ▶ why can't we be more bold?
 - ▶ why is developing confidence about changes in GHC so hard?

Impact

- ▶ alienating new contributors
 - ▶ whole development process and tools focus on core developers
- ▶ backporting is hard (dealing with flaky tests and spurious CI failures)
- ▶ everything becomes quietly coupled, because there's no vision of the systems design
- ▶ innovation stalls, development slows down

The solution

Stable haskell to the rescue



Prior art and related efforts

- ▶ GHC stability principles
- ▶ base split proposal
- ▶ GHC LTS (since 9.14)
- ▶ GHC breakage inventory at <https://github.com/tomjaguarpaw/tilapia>
- ▶ GHC release management document

The solution

- ▶ Focus on the foundations
 - ▶ do not allow shortcuts
 - ▶ fix all the small little deficiencies/debt (coupling, hadrian, bundled mingw, Ways, ...)
 - ▶ each one doesn't seem too serious, but there's a compound effect
- ▶ develop a vision of the systems design
 - ▶ decoupling of the components (base, cabal, hadrian, ...)
- ▶ develop a vision of the user experience
- ▶ establish rigorous quality control
 - ▶ compiling your existing project against a new GHC should be trivial
 - ▶ => more "integration" testing
- ▶ use a modern approach to hosting, CI and contribution

What we are working on

- ▶ Migrating to github
- ▶ Getting rid of Hadrian
- ▶ RTS split
- ▶ cabal-install and cross compilation
- ▶ retargetable GHC
- ▶ GHCup support
- ▶ minimal stage1
- ▶ bootstrap GHC with MicroHS

Deep dive: Getting rid of Hadrian



- ▶ building GHC via cabal
- ▶ some glue code in a Makefile still exists
 - ▶ dealing with inter-stage shenanigans
 - ▶ dealing with bindist creation
 - ▶ we plan to get rid of this too at some point
- ▶ it's a bit slower than hadrian at the moment

Deep dive: RTS split

- ▶ terrible coupling between GHC, hadrian and RTS
 - ▶ `rts.cabal` only describes one library
- ▶ hadrian escape hatches do the rest...
 - ▶ rebuilding the rts for each different flavor (+-threaded, +-debug)
 - ▶ filename conventions and interaction with Ways
 - ▶ bundled libffi copy
 - ▶ knows about per-file flags
 - ▶ ...
- ▶ stable-haskell uses cabal to build GHC and the rts
 - ▶ we have 4 real sublibraries for each flavor
 - ▶ the rts is completely described by the cabal file
 - ▶ requires changes to cabal
 - ▶ GHC can look up the right RTS through the unit ID/package.conf
- ▶ reinstallable RTS?

Deep dive: cabal-install

See #11179

- ▶ adding explicit cross compilation Support
 - ▶ making the solver stage aware (build vs host) so it can solve independently
 - ▶ ability to specify the compiler for build (build-tool-depends) vs host stages
- ▶ removal of “in-place” DB
 - ▶ some ad-hoc nonsense for “local packages”
 - ▶ making it extra hard to package artifacts manually
- ▶ fixing artifacts locations
 - ▶ building now always pre-installs the artifacts into a fake store
 - ▶ we use this to build a bindist

Deep dive: retargetable GHC

- ▶ a cross compiler in stable-haskell is just a package DB
 - ▶ with its own RTS
- ▶ `ghc --target` (same for `ghc-pkg` etc.)
- ▶ we use the same `ghc`
 - ▶ `<target>-ghc` symlinks point to it
 - ▶ if no `--target` switch given, infer the target from `getProgName`
- ▶ thanks to the RTS split, the compiler can pick the rts in the cross package DB

Deep dive: GHCup support

- ▶ GHCup won't add ad-hoc support for alternative compilers
- ▶ only solution: "Installer DSL"
- ▶ Are we package manager yet?
 - ▶ GHCup doesn't need to "know" about 3rdparty tools
- ▶ Agda, Idris, stable-haskell GHC, purescript, ...

Things that need to be done

- ▶ easier bootstrapping
 - ▶ minimal stage1
 - ▶ MicroHS?
- ▶ GHC compilation daemon
 - ▶ alternative to jsem
- ▶ native windows build

Get in touch

- ▶ Github: <https://github.com/stable-haskell>
- ▶ Discord: <https://discord.gg/jWKhhjkSFn>
- ▶ Matrix: [#stable-haskell:matrix.org](https://matrix.org/#stable-haskell)

We'd love to collaborate or hear your ideas!